

Kursens mål

- Du ska klara av att lösa problem på ett strukturerat sätt i programspråket C.

Du ska alltså få en god grund i både problemlösning och C-programmering. Eftersom C är ett språk som är format efter maskinens sätt att arbeta kommer du också att få viss inblick i detta.

Och varför behöver en datalog det? - C-språket är fortfarande mycket vanligt. Det finns också flera scriptspråk som har en imperativ karaktär (perl, php etc). Det finns alltså goda skäl att lära sig imperativ problemlösning såväl som språket C. C är också grunden för (en delmängd av) C++ som också är mycket vanligt. Det är en rimlig allmänbildning att känna till grundläggande datorarkitektur. Vid arbete med t.ex. OS, drivrutiner, mindre system, kompilatorer etc. kommer det är det viktigt att kunna också veta hur maskinen arbetar för att kunna lösa sina problem. För att bli tillräckligt effektiv måste du också läsa en kurs i grundläggande datorarkitektur. Du blir i allmänhet en bättre programmerare (särskilt i C) om du har ett hum om hur maskinen arbetar.

Vad betyder imperativt? - "Imperativ" betyder ungefär "uppmanande" kommer vi kanske ihåg från skolans grammatik. Det är samma betydelse när vi pratar om programmeringsspråk, dvs man bygger upp program med hjälp av satser, där varje sats kan ses som en uppmaning. De tre grundstenarna i den imperativa programmeringsparadigmen,

- Sekvens
- Selektion (eller val, villkor)
- Iteration (eller upprepning, loop)

har ni redan stött på i Java, där de också utgjort en grundsten. Avsaknaden av objekthantering i språket C ger dem dock en större vikt än i Java.

Tekniken att härska genom att söndra (att dela upp större problem i delproblem) har ni använt tidigare, nu kommer det dock att vara mer fokus på att dela upp *bearbetningen* (dvs funktioner) i delar, snarare än att identifiera objekt (även om vi fortfarande kommer att försöka dela upp och gruppera data på ett relevanta sätt, med de medel som står till buds).

Inledande exempel

```
/* This program does not perform anything meaningful, it is
   just a sample to show the syntax of the language C. */
int foo(int n, int x, float r, float pi)
{
    int i, k=1;
    for (i=0; i<n; i++) {
        k = k + r * r * pi;
    }
    if (k >= x)
        return k;
    else
        return k / 2;
}
```

Koden ovan är inte bara ett element i språket C, den är - givet lite andra förutsättningar runt omkring - även giltig Java-kod.

Syntaxen då man betraktar enskilda elementära konstruktioner ska alltså inte behöva ställa till några problem för er som nybörjare. Även betydelsen av sådana språkkonstruktioner är den som ni är vana vid, dvs som man förväntar sig. Det första problemet ni stöter på är att det inte finns lika mycket mer saker än detta att ta till... I C finns inget stöd för att hantera objekt. En god C-programmerare kan med uppjudande av lite kraft ändå i viss utsträckning lösa problem med objektorienterat tänkande. Det är en smaksak var man drar gränsen.

Några punkter där C skiljer sig från Java:

- Det finns inget stöd för objektorienterad programmering i C (klasser saknas)
- För alla identifierare gäller att de får innehålla bara '_', siffror och tecken från engelska alfabetet, dock ej inledande siffror.
- Typer skiljer sig något åt, t.ex. boolean saknas och long är (vanligen) inte 64 bitar.
- C saknar inbyggd stränghantering
- C saknar exceptions
- Ett C-program kan krascha
- C-kod kompileras vanligen till maskinkod
- I C sker kompileringen i två av varandra oberoende steg, preprocessing och därefter den "riktiga" kompileringen.
- Det finns ingen automatisk minneshantering.

Det finns fler saker som skiljer, de får dyka upp under kursens gång. Den skillnad som kommer att påverka din programmering mest är att det inte finns stöd för objektorientering.

Funktioner i C

En funktionsdefinition är den plats där man skriver

- funktionshuvudet med:
 - vilken typ funktionens returvärde har
 - funktionens namn
 - parametrarna till funktionen uppräddade med typ och namn
- koden som ska utföras då funktionen anropas

En funktionsprototyp är funktionshuvudet avslutat med ett ; och där parametrarnas namn ignoreras (de kan alltså utelämnas). Du kan se exempel på prototyper i h-filerna nedan. Betydelsen av en prototyp är att man talar om för kompilatorn att "det finns en funktion någon annanstans som ska anropas på det här sättet" varefter funktionen kan anropas. Kompilatorn kollar så att man anropar med parametrar som till antal och typ stämmer med prototypen. Man kan anropa funktioner utan att det finns en prototyp under förutsättning att själva definitionen finns före anropet (definitionens huvud tjänar då som prototyp).

Ett funktionsanrop har formen
funktionsnamn (parameterlista) ;

och ser alltså ut som i Java bortsett från att det inte finns något objekt i C. Betydelsen är också lika, dvs när programmet körs och exekveringen nått till det ställe där anropet finns kommer funktionen att exekveras med de aktuella parametrarnas värden. Den avgörande skillnaden är att funktionen inte kan påverka variabler i något objekt.

Funktioner som redan finns i C

Eftersom en del saker man behöver göra återkommer ofta, finns det en uppsättning redan definierade funktioner i C. Dessa är samlade i något som kallas ett "lib" (det är inte källkoden som finns där, det är redan kompilerade funktioner). Det finns en standard (den s.k. ANSI-standard) som i detalj beskriver vilka funktioner som ska finnas, för var och en vad den heter, vad den returnerar, vilka parametrar den ska anropas med och vad den ska utföra. Vanligen tillhandahåller utvecklingsmiljöerna ytterligare funktioner som man dock bör undvika att använda om man vill hålla sitt program portabelt, dvs att programmet ska kunna kompileras och köras i någon annan miljö. Redan från början behöver vi kunna mata ett program med data från tangentbordet, och skriva ut de resultatet programmet har räknat fram i ett textfönster. Till vår hjälp har vi då t.ex. funktionerna **`printf`** och **`scanf`**. Exempel på användning av printf:

```
printf("Hello world"); /* Skriver ut texten Hello world */
printf("Hello\nworld"); /* Skriver ut texten Hello på en rad och world på nästa *
printf("Summan är %d kr", 42); /* Skriver ut texten Summan är 42 kr */
printf("Summan är %d kr", sum+23); /* Skriver ut texten Summan är 123 kr om varia
printf("Summan är %d:%d", kr, ore); /* Skriver ut texten Summan är 123:50 om kr h
printf("Summan är %d:%d", ore, kr); /* Skriver ut texten Summan är 50:123 med sam
printf("Summan är %f", sum); /* Skriver ut texten Summan är 123.5 om sum har värd
```

Allmänt

printf måste anropas med minst en parameter. Hur många parametrar utöver den första som den ska anropas med, avgörs av innehållet i den första (som måste vara en sträng). För varje *platshållare* av typen "**%x**" där **x** är någon giltig bokstav för platshållare skall finnas ytterligare en parameter.

Betydelsen är att strängen skrivs ut som den är fränsett att varje platshållare byts ut mot sin motsvarande parameters värde. Platshållare kopplas till parametervärden så att platshållare 1 hämtar värdet från första parametern efter formatsträngen, platshållare 2 hämtar värdet från andra parametern etc.

Några platshållare:

- **%d** skapar en utskrift av bara heltassiffror från ett heltalsvärde
- **%f** skapar en utskrift med en heltalsdel och en decimaldel från ett flyttalsvärde

Det finns vissa möjligheter att mer i detalj specificera vilken form utskriften av ett värde ska få. Exempel på några vanliga:

- **%.2f** begränsar antalet siffror efter decimalpunkten till 2
- **%8d** reserverar *plats* för 8 siffror i textfönstret, skriver ut heltalsvärdet högerjusterat på denna plats och fyller överbliven plats med blanka

I boken kap 2.8 finner du fler platshållare och speciella formatkoder. I kapitel 10.4 finns en fullständig sammanställning. Det är inte meningen att du ska lära dig sådant utantill. Det du behöver förstå nu är att printf gör följande:

- Den antar att det finns en motsvarande parameter.
- Den antar att värdet har den typ som angetts i platshållaren
- Den formatterar enligt anvisningen i platshållaren.

I allmänhet kan kompilatorn inte avgöra om du gjort rätt, dvs om du angett rätt antal parametrar och om var och en av dessa har rätt typ. Ansvar ligger alltså på dig att göra rätt. Om du råkar ange för få parametrar så kan det gå riktigt illa. Detsamma gäller om du råkar ange värden av fel typ. Det senare är problem därför att värden av olika typ kan ha olika storlek (t.ex. upptar värden av typen **char** mindre plats i minnet än värden av typen **double**). Om du t.ex. anger "**%lf %d**", '**x**', **13** så kommer printf att skriva ut ett långt flyttal och sedan leta efter ett heltal som ligger "storleken av ett långt flyttal" längre bort. Då kommer den dock att hamna utanför parameterlistan i minnet, med i princip samma konsekvens som i fallet att man utelämnat en parameter.

Om du kör den rekommenderade miljön KDevelop eller någon annan gcc-baserad miljö, så kommer kompilatorn att varna i alla fall där det är möjligt att avgöra om du gjort fel.

scanf har samma koppling av parametrar till en formatsträng som printf, men används för att ta emot data från tangentbordet. Observera f.a. att om du får för dig att skriva in en ledtext i formatsträngen till scanf, så leder det inte till att strängen skrivs ut.

Viktigt! Varje parameter som ska matcha en platshållare till scanf och är av typen, int, float etc (dvs de typer som vi f.n. kommer i kontakt med), måste föregås av den s.k. adressoperatoren '&'.

Exempel:

```
int x;
```

```
scanf ("%d", &x);
```

Vi kommer snart (föreläsning 2) till betydelsen av adressoperatören...

Typer i C

De datatyper som är inbyggda i C-språket är ungefär desamma som i Java, skillnaden är att boolean saknas. Istället används heltalstyper på varje ställe där man intuitivt förväntar sig en boolean. Värde 0 motsvarar falskt och icke-0 motsvarar sant.

Om att skapa C-program

Återigen: C saknar stöd för objektorienterad programmering och det sätter användningen av funktioner i fokus. Vi måste helt enkelt se till att dela upp problemet i delproblem, lösa dessa var för sig genom att skriva funktioner. Alla funktioner i ett C-program ligger på samma nivå, "platt". Det finns alltså ingen 'this' och det finns inga instansvariabler. Det betyder att det finns goda möjligheter att röra till det när man programmerar, om man inte följer vettiga konventioner. Att göra så hänger på en själv, och vi ska därför jobba på att följa några saker strikt från början.

- Varje program måste ha exakt en funktion som heter **main**. Detta kollar länkaren åt oss.
- Man delar som sagt upp problemet så att det blir fler funktioner, man vill också gruppera alla dessa funktioner i olika *c-filer*.
- Man behöver i allmänhet använda sig av funktioner i standard-libbet (som är en bunt av färdigkompileerade funktioner) som hör till varje utvecklingsmiljö.
- Man kan vilja använda saker från ytterligare något eller några lib som t.ex. någon annan gjort.

För att knyta ihop funktioner som vi själva skrivit och placerat i olika c-filer skriver man s.k. headerfiler (h-filer). Dessa används ett "klistre" mellan en c-fil där funktionen definierats, och c-filer som anropar den. Standardlibbets h-filer används på ett liknande sätt som ett klistre mellan libbet och dina c-filer som anropar funktioner i libbet. Om man gör ett eget lib måste man göra på samma sätt.

Viktigt:

- Om man vill att en eller flera funktioner i en fil ska "exporteras" (dvs. att ge möjlighet att anropa den från funktioner i andra filer), så specificerar man detta i en h-fil (med samma namn som c-filen)
- En c-fil ska **alltid** inkludera "sin egen" h-fil.
- Om en funktion anropas och denna finns i någon annan c-fil (eller lib) så ska relevant h-fil inkluderas. (Det *går* att få tyst på kompilatorn på andra sätt, men vi *ska* använda h-filer).

Om detta följs besparar man sig trassel som kan kosta mycket möda att reda ut.

Exempel på hur det bör se ut (programmet utför inget meningsfullt, det är meningen att du bara ska behöva titta på funktionsdeklarationer och anrop):

```
main.c

#include "f.h"
#include "h.h"
#include <stdio.h> /* Behövs för standardfunktionen printf */
int main(void)
{
    int x=42, y, y1;
    y = f(x);
    if (y>=0)
        y1 = f3(x);
    else
        y1 = f3(x + 1);
```

```
printf("%d", h(y1, y));
return 0;
}
```

f.c

```
#include "f.h"
#include "g.h"
int f3(int a)
{
    return a * a;
}
int f2(int a)
{
    return f3(a - 1) * g(a + 1);
}
int f(int a)
{
    return f2(a) * -13;
}
```

f.h

```
int f3(int a);
int f(int a);
```

g.c

```
#include "g.h"
#include "h.h"
int g(int x)
{
    return x + h(x, 1000);
}
```

g.h

```
int g(int );
```

h.c

```
#include "h.h"
#include <math.h> /* Standardfunktioner för matte */
int h1(int x)
{
    return 4 * x - 3;
}
int h(int x, int y)
{
    return atan(h1(x) + y);
}
```

h.h

```
int h(int, int);
```

Detta är alltså filerna som vi har skrivit. Filerna `stdio.h` och `math.h` behöver man inte veta var de ligger, det vet kompilatorn själv, så vi kringgärdar dessa med `<>` istället för `"`. Några frågor att fundera på:

- Varför inkluderas inte **h.h** i filen **f.c**?
- Varför finns inte prototypen (dvs "malldeklarationen") för funktionen **f2** med i filen **f.h**?
- Varför inkluderar t.ex. **f.c** filen **f.h** (den behövs ju inte för att tillåta anrop av någon funktion, i **f.h** finns bara funktioner som exporteras *från* **f.c**)?
- Varför inkluderar inte alla c-filer alla h-filer, då slipper man ju hålla reda på vilka som ska inkluderas?
- ... eller varför inte slå ihop alla h-filer till en stor?

Vi talar om för kompilatorn att funktionerna som finns i filerna **main.c**, **f.c**, **g.c**, **h.c** tillsammans ska bilda ett program genom att "lägga in dem i ett projekt". När vi senare begär att ett program ska byggas (med "make") så kommer först en s.k. *preprocessor* att startas. Den tar hand om alla s.k.

Om man vill ha med andra lib (som man själv gjort, köpt, fått, hittat på internet, hittat på sin kurshemsida eller som man kommit i besittning av på något annat sätt) så måste man ange detta speciellt till länkaren. Hur detta görs får man kolla i sin respektive utvecklingsmiljö (eller fråga labbassen). För att få kompilatorns tillåtelse att anropa funktionerna i ett sådant libb måste man i de berörda filerna också inkludera den (eller de) h-filer som specificerar vad som finns i libbet. Man brukar då använda include med < > och lägga h-filen i kompilatorns include-katalog. Själva lib-filen brukar man lägga i kompilatorns lib-katalog. Ibland kan någon sorts installationsprogram som följer med libbet se till att fixa så att sakerna hamnar på rätt ställen.

Mer om preprocessorn

Preprocessorn är ofta en del av programmet som kompilerar, men den kör klart sitt "pass" helt innan den "riktiga" kompileringen påbörjas, och blev det fel så startas aldrig den senare. Alla preprocessordirektiv börjar med ett # följt av ett kommando och därefter eventuella parametrar till kommandot. Hela raden är preprocessordirektiv och endast denna. Om raden blir så lång att man behöver fortsätta samma direktiv på en ny, måste man avsluta raden med ett \ tecken. De direktiv vi behöver veta nu är **include** och **define**. include förklarades i förra stycket. define används för att definiera makron. Ett makro kan skapas med och utan parametrar. Först ett exempel utan:

```
#define MAXVALUE 1000
....
    if (n > MAXVALUE)
        printf("Too big number\n");
```

Det fungerar ganska intuitivt, symbolen "MAXVALUE" definieras till texten "1000" i själva definitionen, och innebörden är att varje förekomst av denna symbol byter preprocessor mot texten 1000. För symbolen gäller samma krav som på andra identifierare, men det finns inga krav på texten som den ska bytas mot.

Nu ett exempel med parametrar:

```
#define PRINT_MAX_INT(a, b)    if ((a) > (b)) printf("%d", a); else printf("%d", b)
...
int a=7, b=99;
PRINT_MAX_INT(13, 42);
PRINT_MAX_INT(a+b, xxx);
```

Varje förekomst av `PRINT_MAX_INT` i C-koden måste ha parentser med två parametrar (dvs. det ska helt enkelt se ut som ett funktionsanrop). Dessa `PRINT_MAX_INT(w, u)` där `w` och `u` kan vara vilken text som helst, kommer att ersättas av texten `if ((a) > (b)) printf("%d", a); else printf("%d", b);` varefter alla symboler `a` i denna ersätts med texten `'w'` och alla symboler `b` ersätts med `u`. I exemplet ovan kommer preprocessorns temporärfil att få följande utseende:

```
/*#define PRINT_MAX_INT(a, b)  if ((a) > (b)) printf("%d", a); else printf("%d", b);
```

```
int a=7, b=99;
if ((13) > (42)) printf("%d", 13); else printf("%d", 42);
if ((a+b) > (xxx)) printf("%d", a+b); else printf("%d", xxx);
```

Som du ser blir betydelsen något som påminner starkt om funktionsdefinition respektive funktionsanrop, fast det egentligen bara handlar om textredigering. Då man använder makron är det viktigt att man har klart för sig hur det fungerar.

Det kan vara en god vana att placera alla preprocessor-direktiv upptill i filen. Det kan ibland vara lämpligt att flera c-filer har samma definition av någon symbol. I så fall ska definitionen läggas i en gemensam h-fil.

Utvecklingsmiljön

Det finns flera utvecklingsmiljöer för C-programmering, s.k. IDE:n (integrated development environment). De kan i varierande grad uppfylla de behov en programmerare har:

- Möjlighet att skriva och ändra kod
- Möjlighet att översätta koden till körbar kod (i vårt fall kompilator och länkare).
- Möjlighet att starta och köra programmet direkt från IDE:ns menyer etc (dvs integrerat).
- Möjlighet att felsöka (debugga) sitt program integrerat (dvs att kunna sätta brytpunkter i källkoden, att kunna exekvera enstaka källkodsradar, att kunna granska aktuella värden på källkodens variabler under körning).
- Projekthantering, dvs. funktioner som förenklar för programmeraren om källkoden är fördelad över flera filer.
- Hjälpfunktion integrerat (hjälp både på C-språket och miljön)
- Visa koden i "syntax highlight".
- Automatisk formatering av texten.
- Olika typer av sökfunktioner
- Bygga grafiska användargränssnitt grafiskt (t.ex. med "dra och släpp").

Även om ett IDE uppfyller flera av punkterna ovan, så kan enskilda punkter uppfyllas mer eller mindre bra. Några IDE:n:

- [KDevelop](#), rekommenderas i denna kurs. Körs på linux. [*Fritt program*]
- [Dev C++](#), finns för Windows. [*Fritt program*]
- [LCC](#), finns för Windows [*Fritt program*]
- [RHIDE](#), finns för DOS-baserade Windows (win95, 98 etc. ej NT)[*Fritt program*]
- [Visual C++](#) (Visual Studio) för Windows
- [Borland *](#) för Windows
- [Code Warrior](#) för Windows, MacOS

De rekommenderas i den ordning de står (möjligen att LCC kan anses likvärdig med DevC++, LCC tar betydligt mindre plats och har en bra "resurseditor" men klarar inte C++-utökningen som eventuellt kan vara bra för någon annan kurs, t.ex. C++).